

Day 1 & Day 2 Certification Audit

Machine Learning Engineering — Senior Level Examination

Name: _____

Date: _____

Score: _____ / 100

Total Score: 100 points | Duration: 90 minutes | 25 Multiple-Choice Questions (4 points each)

Select the single best answer for each question. No calculators permitted.

Section	Q1–5	Q6–10	Q11–15	Q16–20	Q21–25
Points	20	20	20	20	20
Score					

Question 1: The StudyGuard Leak

You are building an ML pipeline for a desktop activity monitor to predict a continuous "distraction score" based on webcam pixel intensity. Your code does this:

1. Loads 10,000 rows of video telemetry.
2. Runs `X_scaled = StandardScaler().fit_transform(X)`.
3. Splits `X_scaled` into Train (80%) and Test (20%).
4. Trains `SGDRegressor` and scores an R^2 of 0.95 on the Test set.

When deployed to production, the model fails completely. Why?

- A. `SGDRegressor` cannot handle webcam telemetry data.
- B. Data leakage; the training set had access to the statistical mean and variance of the test set.
- C. You should have used `accuracy_score` instead of R^2 for a distraction score.
- D. The learning rate α was set too high by default.

Question 2: The Retro Reload Expansion

You are predicting continuous future game sales on a digital store database. The residual plot shows that for games predicting under \$5,000 in sales, the errors are within \$500. But for massive AAA games predicting

\$50,000+ in sales, the errors fan out wildly between $-\$15,000$ and $+\$25,000$. What is this sickness called, and what is the fix?

- | | |
|--|--|
| A. Linearity; add a polynomial feature (X^2) to the dataset. | B. Autocorrelation; add a "Release Year" feature. |
| C. Heteroscedasticity; transform the target variable by predicting $\text{np.log}(y_{\text{train}})$. | D. Multicollinearity; use a HuberRegressor to delete the outliers. |

Question 3: The Manual Optimization Engine

Inside a custom for loop executing Gradient Descent, you calculate a prediction of 5.0, but the true target is 12.0. The feature value X is positive. Which of the following describes exactly what the mathematical engine will do to the weight?

$$\begin{aligned} \text{error} &= y_i - \hat{y}_i = 12.0 - 5.0 = 7.0 \text{ (positive error)} \\ \text{gradient} &= \text{error} \times X = \text{positive} \times \text{positive} = \text{positive} \\ \theta &\leftarrow \theta - \alpha \times \text{gradient} \Rightarrow \text{weight decreases} \end{aligned}$$

- | | |
|--|--|
| A. The error is positive, so the gradient is positive, and the weight will decrease. | B. The error is negative, so the gradient is negative, and the weight will increase. |
| C. The error is negative, so the gradient is positive, and the weight will increase. | D. The error is positive, so the gradient is negative, and the weight will decrease. |

Question 4: The Outlier Dilemma

You are analyzing player telemetry. 99% of players log between 1 and 10 hours a week. One specific viral player logs 160 hours. You do not want to delete this data row, but you also don't want it to yank your Linear Regression line into the sky. What is the most efficient, scalable code architecture?

- | | |
|--|--|
| A. <code>LinearRegression()</code> and manually write an if statement to ignore errors over 100. | B. <code>SGDRegressor(loss='squared_error')</code> |
| C. <code>SGDRegressor(loss='huber')</code> | D. Apply <code>np.sqrt(X)</code> to the hours feature to delete the outlier. |

Question 5: The Double Negative

You run a regression model to predict server load. Your final evaluation metrics print out: $R^2 = -0.45$. What does this definitively tell you about your model?

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = -0.45$$

- A. Your model is predicting negative server loads.
- B. Your model is performing exactly 45% worse than it did on the training set.
- C. Your model is completely broken and is actively worse than a dumb baseline that just guesses the mathematical average every time.
- D. You used `accuracy_score` by mistake.

Question 6: SVD vs SGD Reality Check

“I used `LinearRegression` from `sklearn`. It took 1,000 iterations to update the weights for all rows.” What is wrong with this statement?

$$\theta = (X^T X)^{-1} X^T y \quad (\text{Normal Equation — one direct shot, zero iterations})$$

- A. It takes exactly 1 iteration per row, so it should be the length of y .
- B. `Sklearn`’s default `LinearRegression` uses SVD (Singular Value Decomposition) or the Normal Equation, which solves the math in one direct shot with zero iterations.
- C. They should have used Batch Gradient Descent to force it into 1,000 loops.
- D. `LinearRegression` only updates weights once per epoch, so 1,000 iterations means 1,000 epochs.

Question 7: Unstable Matrices

You are predicting frame rates for a C++ console game. You include two features: “Memory allocated in Megabytes” and “Memory allocated in Kilobytes.” You use the Normal Equation. What happens?

```
Memory_MB = [512, 1024, 2048, ...]
Memory_KB = [524288, 1048576, 2097152, ...] // exactly 1024 * Memory_MB
```

These features are perfectly correlated ($r = 1.0$).

- A. The model performs perfectly because it has more data.
- B. The matrix inversion collapses due to perfect multicollinearity, resulting in wildly unstable or completely broken coefficients.
- C. The residual plot will immediately show a megaphone shape.
- D. The Normal Equation will automatically delete the Kilobytes column.

Question 8: The Log Reversal

You executed this perfectly engineered pipeline:

```
y_train_log = np.log(y_train)
model.fit(X_train, y_train_log)
pred = model.predict(X_test)
```

You want to calculate your final RMSE in actual dollars. Which line of code must you write next?

Since $y_{train_log} = \ln(y_{train})$, then $y_{train} = e^{y_{train_log}}$

- A. `actual_pred = pred ** 2`
- B. `actual_pred = np.exp(pred)`
- C. `actual_pred = np.log(pred)`
- D. No action needed; MSE automatically converts logs back to dollars.

Question 9: The Impossible Cost

In your gradient descent loop, your loss history array looks like this after 5 iterations:

```
[14.5, 45.2, 130.8, 412.9, NaN]
```

What is the immediate engineering diagnosis?

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

- A. The features are perfectly scaled.
- B. The learning rate (α) is too high, or the features are unscaled, causing the gradient to overshoot the minimum and diverge.
- C. The test set leaked into the training set.
- D. You forgot to append the column of 1s for the intercept.

Question 10: Evaluating RMSE Context

You are predicting the price of digital games. Your RMSE is \$5.00. The average game price is \$30.00, and the Standard Deviation of game prices is \$2.00. Is this a good model?

$$RMSE = \$5.00, \mu = \$30.00, \sigma = \$2.00$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} = \$5.00$$

- A. Yes, because 5 is a small number compared to 30.
- B. Yes, because the RMSE is under 20%.
- C. No, because your error (\$5.00) is larger than the natural mathematical spread of the data (\$2.00), meaning the model is worse than guessing the mean.
- D. No, because RMSE cannot be measured in dollars.

Question 11: The Shape of Curves

Your residual plot for predicting player matchmaking wait times shows a distinct U-shape curve (the dots dip down below zero in the middle, and curve up on the edges). What is the exact diagnosis and fix?

- A. Autocorrelation; add a Time Index.
- B. Non-linearity; add a polynomial feature like X^2 so the math can bend the line.
- C. Heteroscedasticity; take the `np.sqrt()` of y .
- D. Outliers; switch to `accuracy_score`.

Question 12: The 1/2m Mystery

Why does the formal cost function for Linear Regression include a

$\frac{1}{2}$ at the front of the Mean Squared Error?

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

- A. To cut the final loss metric in half so it looks better to stakeholders.
- B. Pure programming convenience; when you take the calculus derivative, the exponent 2 drops down and cancels out the 1/2, making the gradient math cleaner for the CPU.
- C. It is required to prevent negative numbers from canceling out positive ones.
- D. It converts the prediction from a logarithm back to a straight line.

Question 13: Feature Engineering with 1/X

You are analyzing network ping based on physical distance to the server. Going from 10 miles to 50 miles increases ping massively. Going from 2,000 miles to 2,040 miles barely changes the ping at all. Which feature transformation best captures this “diminishing returns” cliff?

$$f(x) = \frac{1}{x} \quad (\text{Inverse / Reciprocal transformation})$$

- A. X^2
- B. $1/X$
(Inverse/Reciprocal)
- C. `np.sqrt(X)`
- D. `StandardScaler()`

Question 14: NumPy Array Manipulation

In the Normal Equation architecture, you write this code:

```
np.c_[np.ones((X.shape[0], 1)), X]
```

What is the strict architectural purpose of that column of 1s?

$$X_{aug} = [1, x_1, x_2, \dots, x_n]$$

$$\hat{y} = Xw + b \Rightarrow \hat{y} = X_{aug} \cdot \theta \quad (\text{bias absorbed into weight vector})$$

- A. To prevent division by zero in the matrix inversion.
- B. It acts as a mathematical placeholder allowing the matrix multiplication to calculate the Intercept (b) simultaneously with the weights (w).
- C. To scale all the features between 0 and 1.
- D. To manually define the learning rate α .

Question 15: The Confused Classification

“I am predicting whether a user activity on a webcam feed is ‘Studying’ (1) or ‘Distracted’ (0). I trained a Linear Regression model, but the predictions are coming out as 0.74 and 0.22. What went wrong?”

- A. They forgot to apply 1/X to the pixels.
- B. Linear Regression predicts continuous numbers (Regression), not discrete Boolean categories (Classification). They should not be using Linear Regression for this problem.
- C. The learning rate caused the predictions to become decimals.
- D. They need to calculate the Mean Absolute Error to force the numbers to 0 and 1.

Question 16: Accuracy vs R² Reality Check

Why will accuracy_score always return 0.0 (or near zero) when applied to a regression model predicting continuous dollars, even if the model is excellent?

- A. Because R^2 deletes the accuracy score in memory.
- B. Because `accuracy_score` checks for a 100% exact numerical match. A prediction of 100,000.01 compared to an actual 100,000.00 is marked as a complete failure, ignoring how close it was.
- C. Because accuracy only works on matrices with less than 3 columns.
- D. Because the residuals cancel each other out.

Question 17: The MAE Advantage

You are predicting the time it takes to compile a C++ codebase. Usually, it takes 10 seconds. Occasionally, a weird OS hang makes it take 300 seconds. You do not want your optimization engine to panic and aggressively tilt the model just to fix those rare 300-second hangs. Which cost function should you train on?

$$MSE = \frac{1}{n} \sum_{i=1}^n e_i^2 \quad (\text{penalizes outliers heavily})$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |e_i| \quad (\text{penalizes all errors linearly})$$

- | | | | |
|-----------------------------|-----------------------------------|------------------------------|-------------|
| A. Mean Squared Error (MSE) | B. Root Mean Squared Error (RMSE) | C. Mean Absolute Error (MAE) | D. Accuracy |
|-----------------------------|-----------------------------------|------------------------------|-------------|

Question 18: Stochastic vs Batch

What is the fundamental mechanical difference between Batch Gradient Descent and Stochastic Gradient Descent (SGD)?

$$\text{Batch: } \theta \leftarrow \theta - \alpha \cdot \frac{1}{n} \sum_{i=1}^n \nabla J(\theta; x_i, y_i) \quad (\text{one update using all } n \text{ rows})$$

$$\text{SGD: } \theta \leftarrow \theta - \alpha \cdot \nabla J(\theta; x_i, y_i) \quad (\text{one update per row})$$

- A. Batch updates weights 0 times; SGD updates weights 1 time.
- B. Batch calculates the gradient using the entire dataset before making 1 weight update. SGD calculates the gradient using one row at a time, updating the weights thousands of times per epoch.

- C. SGD requires a closed-form Normal Equation; Batch does not.
- D. There is no difference; they are aliases for the same function.

Question 19: Translating Code to Math

If your Python code says:

```
predictions = np.dot(X, weights) + b
```

What is the exact formal mathematical equivalent of this forward pass?

$$A) J(w, b) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$B) \theta \leftarrow \theta - \alpha \nabla J(\theta)$$

$$C) \hat{y} = Xw + b$$

$$D) e_i = y_i - \hat{y}_i$$

- A. $J(w, b) = 1/(2n) \sum (\hat{y}_i - y_i)^2$ B. $\theta \leftarrow \theta - \alpha \nabla J(\theta)$ C. $\hat{y} = Xw + b$ D. $e_i = y_i - \hat{y}_i$

Question 20: The Meaning of the Intercept

You train a model on an IT salary dataset:

```
Salary = w1*(Years Experience) + w2*(Skill Score) + b
```

Your final model outputs $b = \$45,000$. What does this number represent in reality?

When $x_1 = 0$ and $x_2 = 0$:

$$y = w_1(0) + w_2(0) + b = b = \$45,000$$

- A. The average error of the model. B. The base starting salary for someone with exactly 0 Years of Experience and 0 Skill Score.
- C. The maximum possible salary. D. The learning rate multiplied by the gradient.

Question 21: Training Time vs Inference Time

Which of the following data objects is strictly available during Training-Time but mathematically impossible to have during Inference-Time in production?

- A. The Feature vector (X) B. The learned parameters (w and b) C. The exact Target variable (y) D. The Scaling parameters (μ and σ)

Question 22: Transformation Double-Stacking

A dataset has counts of server requests ranging from 1 to 100,000. You apply `np.sqrt(X)` but the residuals still show a cone. To fix it, you decide to write:

```
np.log(np.log(np.sqrt(X)))
```

What is the Senior Engineer's diagnosis of this code?

- A. Perfect; stacking transformations guarantees a perfectly flat residual plot.
- B. Terrible; stacking transformations erases the mathematical gaps between small numbers, destroying the model's ability to differentiate between them. (Occam's Razor: just upgrade to a single $\log(X)$).
- C. It will work, but you must remember to cube the output later.
- D. The Normal Equation requires stacked transformations.

Question 23: The Z-Score Formula

Which equation represents Z-Score Standardization, forcing a feature to have a mean of 0 and a standard deviation of 1?

$$A) \ x' = \frac{x - x_{min}}{x} min$$

$$B) \ z = \frac{x - \mu}{\sigma}$$

$$C) \ P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

$$D) \ d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$$

- A. $x' = (x - x_{\min}) / (x_{\max} - x_{\min})$ [Min-Max Scaling] B. $z = (x - \mu) / \sigma$ [Z-Score Standardization]
- C. $P(A|B) = P(B|A)P(A) / P(B)$ [Bayes' Theorem] D. $d(x, y) = \sqrt{\sum (x_i - y_i)^2}$ [Euclidean Distance]

Question 24: Target vs Feature Transformation

You want to predict percentage changes in house prices, rather than absolute dollar changes, because the price variance grows exponentially. Do you transform the Features (X) or the Target (y)?

- A. Transform the Features (X) using $\log(X)$. B. Transform the Target (y) using $\log(y)$.
- C. Transform both using $1/X$ and $1/y$. D. Do not transform; Linear Regression naturally predicts percentages.

Question 25: The Ultimate Implementation Pattern

According to the Day 2 syllabus and strict workflow protocols, what is the exact chronological order of an ML Pipeline?

- A. Scale Data → Split Data → Train → Establish Baseline → Predict
- B. Split Data → Scale Data (Fit on Train only) → Establish Baseline → Train → Predict → Check Residuals
- C. Establish Baseline → Train → Split Data → Check Residuals → Scale Data
- D. Train → Predict → Scale Data → Split Data

ANSWER KEY

Q1. Answer: B

StandardScaler().fit_transform(X) computes the mean and variance across ALL data before the split. The training set sees test set statistics — classic data leakage.

Q2. Answer: C

Fanning residuals (cone/megaphone shape) = Heteroscedasticity. A log transform on y compresses the large values and stabilizes variance.

Q3. Answer: A

error = 12 - 5 = 7 (positive). gradient = error × X = positive. Weight update: $w - \alpha(\text{positive})$ = decreases.

Q4. Answer: C

Huber loss is MSE for small errors and MAE for large errors — it automatically reduces outlier influence without deleting data.

Q5. Answer: C

$R^2 < 0$ means $SS_{\text{res}} > SS_{\text{tot}}$. The model explains less variance than simply predicting the mean. It is worse than the null baseline.

Q6. Answer: B

LinearRegression uses a closed-form solver (SVD or Normal Equation). There is no iteration loop at all. The junior confused it with SGDRegressor.

Q7. Answer: B

Megabytes and Kilobytes are perfectly collinear (correlation = 1.0). The matrix $X^T X$ becomes singular/non-invertible, destroying the Normal Equation.

Q8. Answer: B

To reverse $\ln(y)$, apply $\exp()$: $\text{actual_pred} = \text{np.exp(pred)}$. Squaring or logging again would be mathematically wrong.

Q9. Answer: B

Loss diverging to NaN = the gradients are exploding. This is caused by a learning rate that is too high or features that have not been scaled.

Q10. Answer: C

RMSE (\$5) > σ (\$2). Your model's average error exceeds the natural spread of the data. It is worse than just guessing the mean every time.

Q11. Answer: B

A U-shape in residuals = non-linearity in the data. The model needs a polynomial feature (e.g., X^2) to capture the curve.

Q12. Answer: B

The 1/2 cancels with the 2 that comes down when you differentiate x^2 . This is purely a calculus convenience to produce a clean gradient.

Q13. Answer: B

1/X (inverse) models diminishing returns: large changes at small X, almost zero change at large X. This matches the ping-distance physics.

Q14. Answer: B

The column of 1s is multiplied by the bias weight θ_0 , computing the intercept. Without it, the regression line is forced through the origin.

Q15. Answer: B

This is a classification problem (binary: Studying vs. Distracted). Linear Regression outputs continuous values. Use LogisticRegression instead.

Q16. Answer: B

accuracy_score checks for exact integer matches. Continuous float predictions like 100000.01 vs 100000.00 are counted as complete misses.

Q17. Answer: C

MAE penalizes all errors linearly regardless of magnitude. MSE/RMSE square the errors, causing the 300-second outliers to dominate the loss.

Q18. Answer: B

Batch: one gradient computation over all n rows $\rightarrow$ one update. SGD: one gradient per row $\rightarrow$ n updates per epoch. Thousands of weight updates per pass.

Q19. Answer: C

$np.dot(X, weights) + b$ is the forward pass $\hat{y} = Xw + b$. Option A is the cost function, B is the update rule, D is the error formula.

Q20. Answer: B

The intercept b is the predicted value when ALL features are zero. It represents the baseline salary for zero experience and zero skill score.

Q21. Answer: C

During inference, you do not have the true labels (y). You only have X (new data), the model parameters (w, b), and the scaler parameters (μ , σ).

Q22. Answer: B

Stacking transformations (sqrt then log then log) destroys the mathematical spacing between small values. Use a single, stronger transform like $\log(X)$ instead.

Q23. Answer: B

$Z = (x - \mu) / \sigma$ produces a distribution with mean = 0 and std = 1. This is the StandardScaler formula.

Q24. Answer: B

To predict percentage changes, you log-transform the TARGET (y), not the features. This changes the loss from absolute to relative error.

Q25. Answer: B

*Correct pipeline: Split first, then scale (fit only on train), establish baseline, train, predict, and check residuals.
Never scale before splitting.*